Compression Representation in OpenL3 and OpenBEATs Embeddings

Griffin Barbieri
Georgia Institute of Technology, Music Informatics Group
gbarbieri6@gatech.edu

***Abstract*—We investigate whether dynamic compression parameter information can be extracted from general-purpose audio embeddings using a contrastive learning framework. Using embeddings from the pretrained models OpenL3 and OpenBEATs, we train four contrastive models on a synthetic audio dataset- two trained with cosine embedding loss, and two with triplet loss. Both OpenL3- and OpenBEATs-based models failed to generalize. Both OpenL3 and OpenBEATs models exhibited representation collapse, with virtually no separation between positive and negative pairs. OpenL3 collapsed immediately, while OpenBEATs collapsed more gradually across training. Linear probes trained on frozen projection head representations at multiple checkpoints confirmed that the projection head learned nothing transferable about compression, with all 16 probes converging on chance accuracy on out-of-distribution Beijing Opera recordings. To establish a lower bound on how much compression signal these embeddings carry, we trained random forest classifiers directly on raw OpenL3 and OpenBEATs embeddings for a binary compressed/uncompressed classification task. Neither classifier generalized to the out-of-distribution evaluation set, with both achieving chance accuracy. Taken together, these results support existing evidence that audio effect parameter information is not straightforwardly accessible from general-purpose audio embedding models, and highlight the need for dedicated architectures or training strategies for AFX representation.**

I. Introduction

Audio style transfer is the task of applying the stylistic characteristics of a reference piece of audio to a target piece of audio. This usually involves matching the sonic character of a professional recording, such as its dynamics and frequency content, and imposing those characteristics onto a separate track. Commercial plugins implementing this approach allow producers to use a reference recording as a target, automatically adjusting processing across dimensions such as dynamic range compression, equalization, and reverberation to bring the target audio in line with the reference. BE-AFX [1] can be understood as a subproblem of this broader task, focused specifically on identifying and transferring the parameters of an effect chain.

In a blind estimation of audio effects (BE-AFX) task, a model or algorithm infers the parameters of an effect chain applied to a piece of audio and reproduces that processing on a separate target signal. Two broad approaches to this problem exist in the literature: reference-based and non-reference-based methods. Reference-based approaches train models directly on parameter loss, which requires ground-truth effect parameters to be available at training time. While this enables precise supervision, it is not always a realistic assumption, as audio datasets frequently lack effect parameter information. Non-reference-based approaches address this limitation by optimizing an audio-domain loss, typically waveform or mel-spectrogram. This makes non-reference methods more practical for training on large, unannotated datasets, at the cost of less direct supervision over specific parameters.

Work by [2] suggests that general-purpose embedding models do not straightforwardly encode audio effect information. Contrastive models are a popular remedy; several studies [3–5] have used contrastive learning to create embedding models that emphasize audio effect information more than general-purpose models such as OpenBEATs or CLAP. However, there is little work done on how contrastive learning can be used to extract the often ambiguous audio effect information from those general models. We used a contrastive learning architecture which we trained to create a similarity space for compression parameters, then explored what specific compression information is represented in OpenL3 [6, 7] and OpenBEATs [8, 9] audio embeddings. This paper highlights a need for more approaches to extract AFX parameter information from general audio and music embedding models, and offers insight into how a contrastive learning approach can potentially be improved.

II. Related Work

BE-AFX has emerged as an area of active interest within the rapidly growing ecosystem of machine learning tools for music production. Early DSP-based approaches to BE-AFX relied on strict mathematical assumptions about the structure of audio effects, resulting in algorithms that lack robustness across variations of nominally similar effects. For instance, two distortion plugins implementing slightly different clipping algorithms may defeat a system designed around a specific mathematical model of distortion [10]. Advances in machine learning over the last decade have enabled a range of more flexible approaches to this problem.

One prominent approach is differentiable digital signal processing (DDSP), which models audio effects as differentiable processes that can be integrated directly into neural network training pipelines [11]. A key advantage of this formulation is that the model learns how each parameter affects the output signal, rather than implicitly inferring parameter interactions from data. This makes DDSP particularly well-suited to non-linear effects, where inter-parameter relationships are complex.

Another approach employs neural proxies of audio effects, in which a neural network learns to generate audio that matches the dynamic and spectral characteristics of target audio effects. This has been used effectively in work such as [1, 4] to synthesize audio with specified effect characteristics. However, neural proxy approaches typically operate outside of real-time production contexts. A guitarist using [4], for instance, cannot interactively adjust their tone after generation; altering any effect parameter requires regenerating the clip entirely. This limits their practical utility in iterative music production workflows.

[12] presents a paradigm that addresses this limitation by enabling style transfer using the user's own audio effects. Given a reference piece of audio, the model optimizes the parameters of the user's effect chain to match the processing applied to the reference as closely as possible. This has a particularly practical downstream use in music production; a producer could match the dynamics and frequency content of their track to a reference recording using their own tools, and then fine-tune the resulting effect parameters directly. To do this, they created a "style similarity metric" based on a novel embedding model for the purposes of inference-time parameter matching. They write that this space is effective for matching the parameters of arbitrary audio effect chains, including non-linear effects. In previous work, [2] established a need for this purpose-built model by showing evidence that audio effect information is not easily extractable from general-purpose audio embedding models, such as OpenBEATs and CLAP. However, we want to examine what non-linear parameter information can be extracted with openly-available audio models, namely OpenL3 [7] and OpenBEATs [8, 9]. To do so, we use a contrastive learning framework using OpenL3 and OpenBEATs embeddings to build a similarity space for dynamic compression parameters.

III. Methods

We train a contrastive learning model directly on OpenL3 [7] and OpenBEATs [9] embeddings, as there is little literature on using contrastive learning to extract ambiguous effect information from existing general-purpose models. OpenL3 was selected for its high-dimensional output; OpenBEATs was included because its 20ms window may better capture the ms-scale timing details of dynamic compression that a 1-second aggregate embedding could obscure. We trained models with both cosine embedding loss and triplet loss to compare objectives. Linear probes at multiple checkpoints let us test whether the projection head learned any generalizable compression representation, even if validation metrics showed no separation. Finally, a random forest classifier on the raw embeddings establishes a lower bound on how much compression signal is accessible before any learned transformation.

A. Inputs and Flowcharts

IV. Design

A. Methodology

Our study proceeded in three experiments, each motivated by the results of the previous.

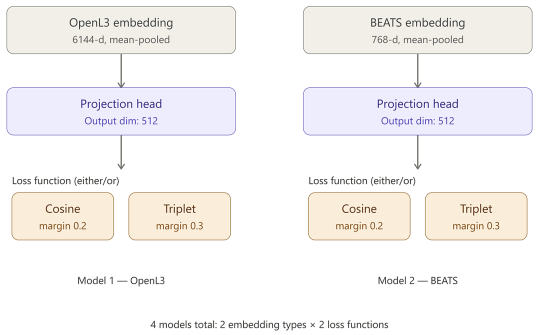


Fig. 1: Contrastive learning model architecture. Temporally-averaged OpenL3 or OpenBEATs embeddings are passed through a 512-dimensional projection head, trained with either cosine embedding loss or triplet loss.

Experiment 1: Contrastive learning. We first attempted to build a similarity space over dynamic compression parameters using a contrastive learning framework applied directly to pretrained audio embeddings. We trained the model on OpenL3 and OpenBEATs embeddings separately, using both cosine embedding loss and triplet loss, yielding four models in total. Each audio clip was embedded into a series of frame-level vectors and mean-pooled into a single vector before being passed to the contrastive model. Mean pooling was selected as the aggregation strategy because it preserves the closest approximation to the original embedding space while incorporating information from the full clip. OpenL3 [7] produces 6,144-dimensional embeddings with a 1-second window and 500ms hop; OpenBEATs [8, 9] produces 768-dimensional embeddings with a 20ms window. After OpenL3 embeddings resulted in poor similarity discrimination by the projection head, we included OpenBEATs to investigate whether its finer temporal resolution would yield richer representations of dynamic compression, whose attack and release parameters operate on the order of milliseconds.

Experiment 2: Linear probing. Validation metrics from the contrastive models showed no separation between positive and negative pairs, suggesting the projection head had not learned a useful compression representation. To determine whether any compression-relevant structure had been learned at any point during training, we froze the projection head weights at multiple checkpoints and trained linear probes directly on its intermediate representations. Probes were trained and evaluated on the out-of-distribution Beijing Opera test set, classifying recordings as lightly or heavily compressed. We used a k-fold approach with 5 folds and 1,000 training epochs per fold, reporting mean train and validation accuracy at convergence across folds (mean $\pm$ std) for each checkpoint. Using a temporal average approach to get a single vector per clip, there was no risk of data leakage.

Experiment 3: Raw embedding classification. Linear

probe results confirmed that the projection head had not learned a useful compression representation. To establish how much compression signal exists in the raw embeddings prior to any learned transformation, we trained random forest classifiers directly on mean-pooled OpenL3 and OpenBEATs embeddings for a binary compressed/uncompressed classification task. This experiment served as a lower bound: if compression signal is not recoverable even by a classifier operating directly on the embeddings, it is unlikely that a contrastive framework acting on the same embeddings could succeed.

B. Dataset

Our contrastive models were trained on a purpose-built synthetic dataset derived from the Lakh MIDI Dataset [13]. MIDI clips were rendered to audio using soundfonts drawn randomly from a pool of 500, producing diverse timbral variation across examples. After normalization, we applied dynamic compression stochastically to each rendering using a parametric compressor. To ensure that compression artifacts were perceptually meaningful, we only retained renderings where the difference signal between compressed and uncompressed audio exceeded -60 dBFS for a cumulative duration of at least one second.

Each training example is structured as a triplet (A, B, C); the full dataset construction pipeline is illustrated in Appendix Fig. 5. Clips A and B are renderings of two distinct MIDI files using two distinct soundfonts, both compressed with identical parameter settings. Clip C shares the same MIDI source and soundfont as clip A, but is compressed with a markedly different parameter set. This triplet structure is designed to train the model to associate A and B by compression similarity while contrasting A and C by compression dissimilarity independent of timbral identity.

We extract embeddings from this audio using two pretrained audio embedding models: OpenL3 and OpenBEATs. OpenL3 was selected initially for its high-dimensional 6,144-dimensional output, which allows it to represent a large amount of acoustic information per frame. However, its 1-second analysis window is coarse relative to the timescales of dynamic compression; attack and release parameters operating on the order of milliseconds may not be prominently represented in a 1-second aggregate embedding. We therefore additionally extracted embeddings using OpenBEATs [9], which operates with a 20 ms window, providing a much higher temporal resolution and finer-grained tracking of the moment-to-moment dynamics that compression acts upon. Both embedding sets were extracted from the same underlying audio dataset; the two models were then trained separately, each on their respective embeddings.

We also constructed a small out-of-distribution test set of Beijing Opera audio [14]. Beijing Opera was selected as the out-of-distribution evaluation domain because its recordings are melodic, largely monophonic, and timbrally homogeneous — properties shared with the MIDI-rendered training audio — while differing substantially in timbre and note event envelope, providing a controlled test of cross-domain generalization. These recordings consisted of solo vocal and instrumental performances of Beijing Opera excerpts. Each recording was randomly compressed either ”heavily” (normalized parameter values 0.7–1.0) or ”lightly” (normalized parameter values 0.0–0.3). These ranges were chosen to introduce variance within each class while maintaining a clear gap in the middle of the parameter space, ensuring that the two classes were meaningfully distinct. This dataset was used both for the linear probe evaluation and for the RF classifier evaluation, yielding 389 embeddings total.

For the classification task, a separate dataset was generated; details are provided in Appendix VII.

C. Contrastive Model Training

Four contrastive learning models were trained in total: two operating on OpenL3 embeddings and two operating on OpenBEATs embeddings. Prior to training, all embeddings were temporally averaged across frames into a single vector per clip, then batch normalized across the training set to zero mean and unit variance. For each embedding type, the first model was trained using cosine embedding loss, implemented via PyTorch’s built-in `CosineEmbeddingLoss`, randomly selecting a positive pair (A, B) or a negative pair (A, C). PyTorch handles L2 normalization before calculating loss. The second model was trained using triplet loss, using the full (A, B, C) triplet training example to pull A closer to B than to C in the learned embedding space. All four models employed a 2-layer MLP projection head with an output dimensionality of 512. No dropout was applied to the OpenL3 models. To investigate whether representation collapse in the OpenBEATs models could be mitigated, we trained those models with dropout rates of 0, 0.2, 0.5, and 0.8; none of these changed the collapse behavior. These dropout rates only slowed OpenBEATs’ rate of collapse. Full contrastive learning architecture details can be found in Appendix Table VI.

D. Random Forest Training

Following inconclusive linear probe results, we turned to a pure classification task as a direct test of whether compression signal is preserved in OpenL3 and OpenBEATs embeddings. For each clip in the classification dataset, frame-level embeddings were averaged across time into a single vector, and these mean embeddings were used as input features to a random forest classifier trained to distinguish compressed from uncompressed clips. This experiment tested how directly compression information is represented in OpenL3 and OpenBEATs embeddings prior to any learned transformation. Training used a pair-aware 80/20 split (via `GroupShuffleSplit`) to prevent source-audio leakage, ensuring both clips from each MIDI pair always fell in the same partition. The “Train Acc.” reported in Appendix Table IV is the accuracy on the internal 20% held-out portion of the training data. Classifier hyperparameters are listed in Appendix Table V.

E. Tests

Validation loss in our contrastive model exhibited small pockets of stability at specific epochs, and to determine

whether these represented any meaningful learned representations, linear probes were trained on checkpoints drawn from these epochs. 16 linear probes were trained in total: 8 on OpenBEATs-based contrastive model checkpoints at epochs 3, 65, 200, and 300, and another 8 on OpenL3-based checkpoints at the same epochs. For each linear probe, the projection head weights were frozen at the relevant checkpoint, and the probe was trained on its representations of the Beijing Opera test set [14], classifying recordings as lightly or heavily compressed. This out-of-distribution test set was also used in evaluating the random forest classifier.

V. Results

A. Summary

B. Contrastive Learning Results

1) OpenL3: Both the cosine and triplet loss variants of the OpenL3-based contrastive model showed no meaningful separation between positive and negative pairs (Appendix Table III). Under cosine embedding loss, positive and negative similarities were essentially indistinguishable by epoch 150 and remained so through epoch 300. The triplet loss variant similarly showed positive and negative distances converging to near zero by epoch 150 and remaining there through epoch 300. Complete per-epoch metrics are reported in Appendix Table III.

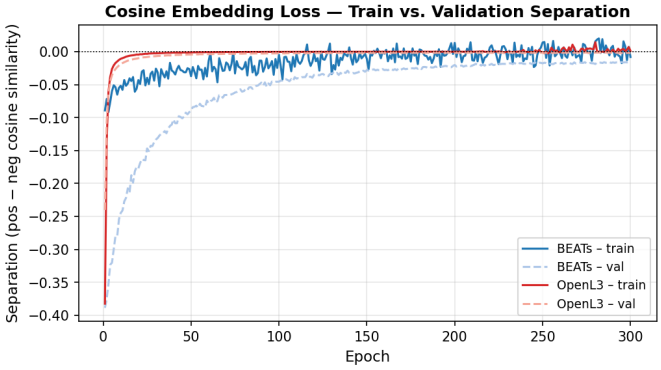


Fig. 2: Training and validation separation (positive minus negative cosine similarity) over 300 epochs for cosine embedding loss models. A positive value indicates successful learning; all models remain at or below zero throughout training.

2) OpenBEATs: We did not use the same training strategy for OpenBEATs as for OpenL3, so the two models' performances are not directly comparable. The OpenBEATs-based contrastive models showed the same pattern as OpenL3, but more gradually. OpenBEATs models were trained with dropout rates of 0.2, 0.5, and 0.8, and more pairs per epoch (Appendix Table III) while no dropout was applied to the OpenL3 models. Thus the slower convergence is likely attributable at least in part to dropout rather than any difference in embedding content. Under cosine embedding loss, training positive and negative similarities moved slowly closer together over 300 epochs, with validation similarities near ceiling throughout. Under triplet loss, training distances narrowed from epoch 1 onward while validation positive and negative distances both converged to near zero early in training. Complete per-epoch metrics are reported in Appendix Table III.

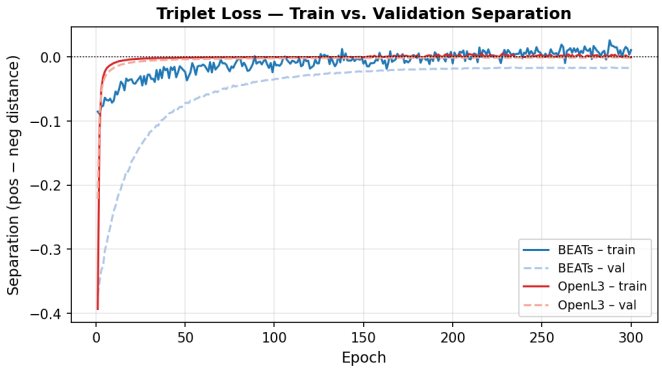


Fig. 3: Training and validation separation (positive minus negative distance) over 300 epochs for triplet loss models. Both models exhibit representation collapse, with separation remaining at or below zero throughout. OpenL3 collapses immediately while OpenBEATs collapses more gradually, with train separation briefly trending upward before converging near zero.

C. Linear Probe

After failing to build an effective similarity space, we froze the projection head weights at multiple checkpoints and ran linear probes to determine whether it had learned any generalizable representation of compression (Appendix Fig. 4). Linear probes were trained on frozen projection head representations drawn from checkpoints at epochs 3, 65, 200, and 300 for all four contrastive models, yielding 16 probes in total. Each probe was evaluated on 389 out-of-distribution Beijing Opera recordings compressed either lightly (normalized parameter values 0.0–0.3) or heavily (0.7–1.0), using 5-fold cross-validation over 1,000 probe training epochs. Across all 16 conditions, mean validation accuracy at probe convergence ranged from 42.5% to 51.7%, with no checkpoint, model type, or loss function producing results meaningfully above chance. Neither light nor heavy compression was reliably distinguished: per-class accuracies fluctuated around 50% without consistent direction, and in several cases the probe settled into a degenerate solution predicting only one class (e.g., the OpenL3 cosine model at epoch 3 achieved 100% light accuracy and 0% heavy accuracy at probe epoch 1, converging to 45.1%/48.6% by epoch 1000). Results at epoch 3 and epoch 300 were almost indistinguishable, consistent with the projection head having learned nothing transferable about compression at any point during training. Full per-checkpoint, per-class accuracy results are reported in Appendix Table II; probe setup details are in Appendix VII.

D. Classification Results

Since we could not find a clear compression signal from the contrastive learning projection head, we wanted to assess how strongly compression is encoded in the raw embeddings themselves. We pivoted to a direct binary classification task,

compressed vs. uncompressed, on an out-of-distribution evaluation set of 389 Beijing opera stems. We used Random Forest classifiers trained separately on OpenBEATs (768-dim) and OpenL3 (6144-dim) embeddings generated from the Lakh MIDI dataset and a pool of 500 soundfonts.

Neither classifier generalized to the OOD set. OpenBEATs achieved 50.0% overall accuracy, indistinguishable from chance, and exhibited a strong bias toward predicting "compressed": it correctly identified 76.8% of compressed clips but only 24.1% of uncompressed ones. OpenL3 performed similarly poorly at 47.4%, with the opposite bias. It correctly classified 69.0% of uncompressed clips, but only 25.0% of compressed ones. The macro-averaged F1 scores of 0.465 and 0.445 for OpenBEATs and OpenL3 respectively confirm that neither model learned a generalizable representation of compression. The opposing class biases further suggest that both classifiers latched onto dataset-specific distributional artifacts rather than any generalizable signature of the compression. Full classification results are reported in Appendix Table IV.

VI. Discussion

A. Limitations

In our contrastive learning triplet dataset, we did not normalize any training audio after compression, resulting in our models receiving both loudness and compression signal in training and potentially allowing them to learn loudness as a proxy for compression. Our batch normalization approach may have contributed to representation collapse by allowing the projection head to exploit batch-level statistics rather than learning genuine compression similarity, consistent with known interactions between BatchNorm and contrastive objectives [15].

B. Discussion

Though [2] show evidence that audio effect transformations have weak signal in embeddings from models like OpenBEATs and CLAP, or are otherwise encoded in a non-straightforward way, we sought to extract that effect information using a contrastive learning framework. We reasoned that such an approach might be effective in addressing the non-linearity of dynamic compression and extracting parameter information even when encoded non-trivially. We found no evidence that specific compression parameter information can be extracted from OpenL3 or OpenBEATs embeddings with this framework. Our results do not support our hypothesis that contrastive learning is an effective architecture for BE-AFX based on embeddings from pretrained models. Our contrastive learning models converged to poor separations, with both OpenL3 and OpenBEATs projection heads exhibiting representation collapse. Linear probes run on the projection head at multiple checkpoints showed no evidence of it learning generalizable compression representations. Lastly, a simple random forest classifier was unable even to distinguish compressed from uncompressed audio.

These results support existing evidence that many existing audio embedding models do not encode significant audio effect information [2][12]. There are several caveats worth investigating, however. Temporally averaged input vectors could represent specific compression information in [12]. In OpenL3 or OpenBEATs embeddings, however, where audio effect information is not encoded in a straightforward way, temporal averaging possibly eliminates subtle timing relationships that may have contained that information. To preserve relevant audio information and timing factors, it may be worthwhile to investigate substituting a GRU for the averaging approach. A two-way GRU might better preserve timing information and amplitude crests across audio windows. This may more effectively represent the action of the compressor, giving the projection head better information about transient shapes and the attack and release of the effect. However, given our classification results for compressed/uncompressed audio, it is also possible that detailed transient shaping is not represented in any useful capacity by OpenL3 or OpenBEATs.

C. Contribution

This paper highlights a need for more approaches to extract AFX parameter information from general audio embedding models, and offers insight into how a contrastive learning approach can potentially be improved. Any tools based on these approaches would have myriad uses in music production and audio editing contexts. Several proprietary style transfer VST plug-ins exist, often based on novel architectures similar to that illustrated in [12, 16]. Finding ways to extract parameter information from pretrained models such as OpenL3 and OpenBEATs may make it easier to develop new tools for style transfer and related tasks.

VII. Conclusion

We investigated whether dynamic compression information can be extracted from general-purpose audio embeddings using contrastive learning. Using embeddings from the pretrained models OpenL3 and OpenBEATs, we trained four contrastive models on synthetic audio, and found that no model could generalize to an OOD test set. We experimented with both cosine embedding loss and triplet loss for our models, but all four models exhibited representation collapse — immediately in the case of OpenL3, and more gradually for OpenBEATs. Linear probes trained on frozen projection head representations at multiple checkpoints confirmed that the projection head learned nothing transferable about compression, with all 16 probes converging on chance accuracy on out-of-distribution Beijing Opera recordings. To establish a lower bound on how much compression signal these embeddings carry, we trained random forest classifiers directly on raw OpenL3 and OpenBEATs embeddings for a binary compressed/uncompressed classification task. Neither classifier generalized to the out-of-distribution evaluation set, with both achieving chance accuracy. Taken together, these results support existing evidence that audio effect parameter information is not straightforwardly accessible from general-purpose audio embedding models, and highlight the need for dedicated architectures or training strategies for AFX representation.

REFERENCES

[1] C. Peladeau and G. Peeters, "Blind estimation of audio effects using an auto-encoder approach and differentiable digital signal processing," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 856–860.

[2] S. H. Hawley and C. J. Steinmetz, "Leveraging neural representations for audio manipulation," in *Proceedings of the 154th Convention of the Audio Engineering Society*, Espoo, Finland, May 2023, aES Convention Paper 41.

[3] J. Koo, M. A. Martínez-Ramírez, W.-H. Liao, S. Uhlich, K. Lee, and Y. Mitsufuji, "Music mixing style transfer: A contrastive learning approach to disentangle audio effects," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[4] Y.-H. Chen, Y.-T. Yeh, Y.-C. Cheng, J.-T. Wu, Y.-H. Ho, J.-S. R. Jang, and Y.-H. Yang, "Towards zero-shot amplifier modeling: One-to-many amplifier modeling via tone embedding control," in *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, CA, USA, November 2024, pp. 446–453.

[5] A. Wright, A. Carson, and L. Juvela, "Open-amp: Synthetic data framework for audio effect foundation models," in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.

[6] R. Arandjelovic and A. Zisserman, "Look, listen and learn," in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 609–617.

[7] J. Cramer, H.-H. Wu, J. Salamon, and J. P. Bello, "Look, listen, and learn more: Design choices for deep audio embeddings," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, UK, May 2019, pp. 3852–3856.

[8] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, "BEATs: Audio pre-training with acoustic tokenizers," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, July 2023, pp. 5178–5193.

[9] S. Bharadwaj, S. Cornell, K. Choi, S. Fukayama, H.-j. Shim, S. Deshmukh, and S. Watanabe, "OpenBEATs: A fully open-source general-purpose audio encoder," 2025. [Online]. Available: https://arxiv.org/abs/2507.14129

[10] J. T. Colonel, M. Comunità, and J. D. Reiss, "Reverse engineering memoryless distortion effects with differentiable waveshapers," October 2022.

[11] C. J. Steinmetz, J. Pons, S. Pascual, and J. Serrà, "Automatic multitrack mixing with a differentiable mixing console of neural audio effects," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, Canada, June 2021, pp. 71–75.

[12] C. J. Steinmetz, S. Singh, M. Comunità, I. Ibnyahya, S. Yuan, E. Benetos, and J. D. Reiss, "ST-ITO: Controlling audio effects for style transfer with inference-time optimization," in *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, CA, USA, November 2024.

[13] C. Raffel, "Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and matching," Ph.D. dissertation, Columbia University, 2016.

[14] R. Gong, R. C. Repetto, and X. Serra, "Creating an a cappella singing audio dataset for automatic Jingju singing evaluation research," in *Proceedings of the 4th International Workshop on Digital Libraries for Musicology (DLfM)*. Shanghai, China: Association for Computing Machinery, 2017, pp. 37–40. [Online]. Available: https://doi.org/10.1145/3144749.3144757

[15] Y. Wu and J. Johnson, "Rethinking "batch" in BatchNorm," May 2021.

[16] C. J. Steinmetz, N. J. Bryan, and J. D. Reiss, "Style transfer of audio effects with differentiable signal processing," AES E-Library, 2022. [Online]. Available: https://aes.org/publications/elibrary-page/?id=21883

APPENDIX

TABLE I: Compression parameter ranges used in pair generation.

Parameter	Min	Max	Unit	Sampling
Threshold	-20	-1	dBFS	uniform continuous
Ratio	1.2	20	—	uniform continuous
Attack	0.01	30	ms	uniform continuous
Release	0.01	30	ms	uniform continuous
Knee width	0	12	dB	uniform discrete (integer)

Linear Probe Setup: 16 linear probes were trained in total: 8 on OpenBEATs-based contrastive model checkpoints at epochs 3, 65, 200, and 300, and another 8 on OpenL3-based checkpoints at the same epochs. Each probe used 5-fold cross-validation over 1,000 training epochs, evaluated on frozen projection head representations of the Beijing Opera OOD test set, classifying recordings as lightly or heavily compressed.

TABLE II: Linear probe accuracy (%) at probe convergence (epoch 1000), 5-fold cross-validation. Values shown as mean $\pm$ std across folds. All results are near chance (50%). Light = lightly compressed class; Heavy = heavily compressed class.

Embedding	Loss	Checkpoint	Train Acc. (%)	Val. Acc. (%)	Light Acc. (%)	Heavy Acc. (%)
OpenL3	Cosine	3	49.8 ± 2.8	46.8 ± 5.9	53.8	55.3
		65	52.1 ± 4.1	42.5 ± 5.0	60.3	41.4
		200	49.6 ± 1.3	46.3 ± 1.8	58.9	36.8
		300	51.0 ± 1.6	44.8 ± 4.8	55.8	43.4
OpenL3	Triplet	3	49.9 ± 2.7	51.5 ± 6.3	46.8	47.4
		65	50.1 ± 2.1	46.8 ± 7.7	56.4	46.1
		200	49.6 ± 2.7	50.7 ± 5.6	55.1	38.2
		300	51.6 ± 2.5	46.6 ± 7.5	64.1	45.4
OpenBEATs	Cosine	3	50.6 ± 1.8	49.4 ± 3.6	51.3	45.4
		65	52.5 ± 2.6	47.3 ± 3.5	53.8	53.3
		200	49.3 ± 2.9	46.6 ± 6.4	43.6	50.0
		300	51.3 ± 2.4	50.4 ± 3.8	46.2	50.0
OpenBEATs	Triplet	3	49.9 ± 3.6	50.9 ± 3.4	51.3	47.4
		65	49.0 ± 1.5	51.7 ± 5.3	49.4	46.7
		200	49.4 ± 1.7	45.8 ± 2.3	45.5	49.3
		300	49.7 ± 1.3	46.0 ± 2.6	46.2	51.9

TABLE III: Contrastive model metrics at selected training epochs. Cosine similarity reported for cosine embedding loss models; mean distance reported for triplet loss models. Positive pairs should score higher similarity / lower distance than negative pairs for successful learning. Sep = positive minus negative.

Model	Loss	Epoch	Train			Validation		
			Pos	Neg	Sep	Pos	Neg	Sep
OpenL3	Cosine	3	0.937	0.997	-0.061	0.937	1.000	-0.063
		65	0.999	1.000	-0.001	0.996	1.000	-0.004
		200	1.000	1.000	<0.001	0.998	1.000	-0.002
		300	0.999	0.998	$+0.001$	0.998	1.000	-0.002
OpenL3	Triplet	3	0.051	0.000	-0.050	0.054	0.000	-0.054
		65	0.001	0.000	<0.001	0.003	0.000	-0.003
		200	0.000	0.000	<0.001	0.001	0.000	-0.001
		300	0.001	0.000	<0.001	0.001	0.000	-0.001
OpenBEATs	Cosine	3	0.145	0.237	-0.092	0.646	0.998	-0.352
		65	0.329	0.376	-0.047	0.925	1.000	-0.075
		200	0.477	0.464	$+0.013$	0.980	1.000	-0.020
		300	0.479	0.487	-0.008	0.985	1.000	-0.014
OpenBEATs	Triplet	3	0.866	0.789	-0.077	0.335	0.002	-0.333
		65	0.659	0.635	-0.024	0.056	0.000	-0.056
		200	0.526	0.516	-0.010	0.018	0.000	-0.018
		300	0.526	0.536	$+0.011$	0.017	0.000	-0.017

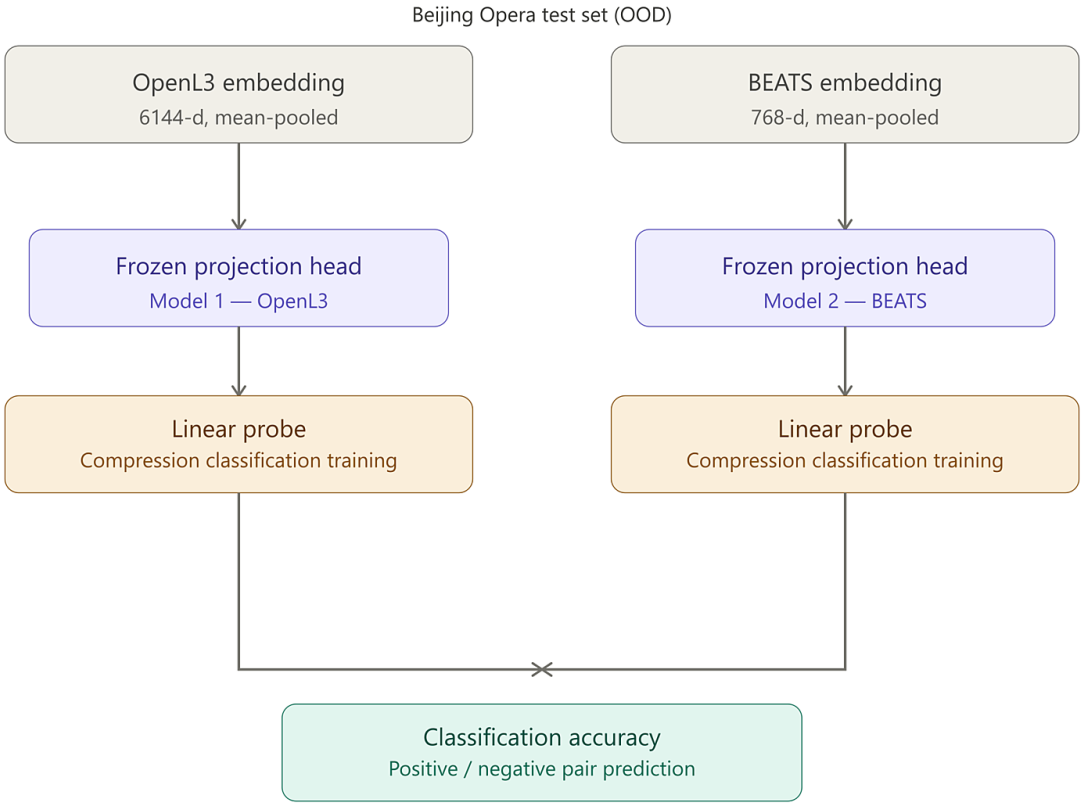


Fig. 4: Linear probe evaluation pipeline. Projection head weights are frozen at a given checkpoint; its representations of out-of-distribution Beijing Opera recordings are passed to a linear classifier trained to predict compression level (light vs. heavy).

Classification Dataset: For the classification task, a separate dataset was generated. Each pair consists of two audio clips rendered from the same MIDI file with the same soundfont, both amplitude-normalized before compression, with random compression applied to one clip and the other left uncompressed. The compressed clip from each pair was re-normalized after compression to ensure the classifier could not learn any gain information. This design represents a best-case scenario for the classification task, isolating compression as the sole distinguishing signal between clips by eliminating timbral and source variation. 313 such pairs were generated for each embedding model, yielding 626 clip total.

TABLE IV: Random forest classifier results on the OOD Beijing Opera test set (114 clips). Train accuracy shown on the MIDI-rendered training set.

Model	**Train Acc.**	**OOD Test Accuracy**			**Macro F1**
		Overall	Uncompressed	Compressed	
OpenBEATs	59.5%	50.0%	24.1%	76.8%	0.465
OpenL3	55.0%	47.4%	69.0%	25.0%	0.445

TABLE V: Random forest classifier hyperparameters.

Hyperparameter	**Value**
`n_estimators`	300
`max_depth`	None (fully grown)
Splitting criterion	Gini impurity (default)
Feature subsampling	$\sqrt{d}$ features per split (`max_features="sqrt"`)
Train/test split	80/20 pair-aware (`GroupShuffleSplit`, `random_state=42`)

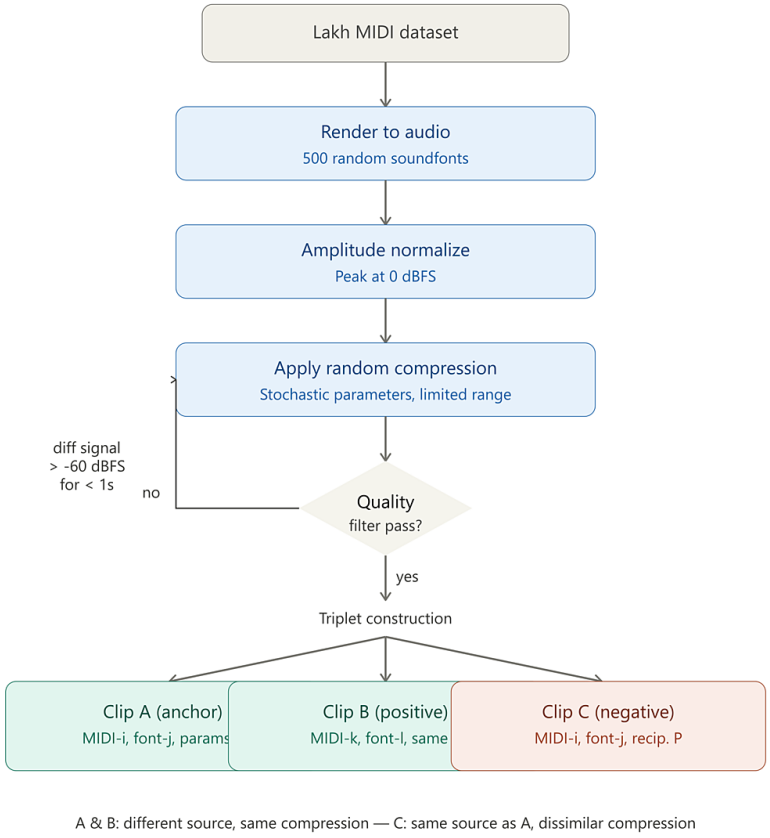


Fig. 5: Training dataset construction pipeline. MIDI clips are rendered to audio using randomly sampled soundfonts, normalized, compressed with stochastically sampled parameters, and quality-filtered. Valid clips are structured into triplets (A, B, C), where A and B share compression parameters across different sources and C is the same source as A compressed with the complementary parameter set.

TABLE VI: Projection head architecture and training hyperparameters for each embedding model.

Property	**OpenL3**	**OpenBEATs**
Architecture		
Input $\rightarrow$ projection dim	$6144 \rightarrow 512$	$768 \rightarrow 512$
Hidden dim	512	512
Layers	2	2
Activation	ReLU	ReLU
Normalization (hidden)	BatchNorm1d	BatchNorm1d
Dropout	none	$p = 0.5$
Output normalization	L2 (unit sphere)	none
Optimizer		
Algorithm	Adam	Adam
Learning rate	1×10^{-4}	1×10^{-5}
β_1, β_2	0.9, 0.999 (PyTorch default)	0.9, 0.999 (PyTorch default)
ε	1×10^{-8} (PyTorch default)	1×10^{-8} (PyTorch default)
Weight decay	none	none
LR scheduler	none	none
Gradient clipping	none	none
Random seed	42	42
Training		
Epochs (cosine / triplet)	300 / 300	300 / 300
Batch size	64	256
Pairs per epoch (train)	800	4,000
Val fraction	0.20	0.20
Cosine embedding loss margin	0.2	0.1
Triplet loss margin	0.3	0.3